

Inventory Management System

Development Standards & Guidelines

This PDF is a professionally formatted version of the project guidelines.

Inventory Management System — Project Guidelines

1. Naming Conventions

C# Classes & Interfaces

Element	Convention	Example
--- --- ---		
Class	PascalCase	`ProductService`, `CategoryRepository`
Interface	`I` prefix + PascalCase	`IProductService`, `ICategoryRepository`
Abstract class	PascalCase (no prefix)	`BaseRepository`
Static class	PascalCase	`SlugHelper`
Enum	PascalCase	`UserRole`

ASP.NET Core Layers

Type	Suffix	Example
--- --- ---		
Controller	`Controller`	`ProductsController`
Service Interface	`I` prefix + `Service` suffix	`IProductService`
Service Implementation	`Service` suffix	`ProductService`
Repository Interface	`I` prefix + `Repository` suffix	`IProductRepository`
Repository Implementation	`Repository` suffix	`ProductRepository`
Request DTO	`Request` suffix	`CreateProductRequest`
Response DTO	`Response` suffix	`ProductResponse`
DbContext	`DbContext` suffix	`AppDbContext`
Middleware	`Middleware` suffix	`RoleMiddleware`

Database

Element	Convention	Example
--- --- ---		
Table	snake_case (plural)	`product_masuk`, `product_keluar`
Column	snake_case	`category_id`, `remember_token`
Primary Key	`id`	`id`
Foreign Key	{table}_id	`product_id`, `supplier_id`

| Pivot tables | singular_singular | `product\_category` |

API Routes

- Use **kebab-case** (lowercase with hyphens)
- Plural nouns for resources
- No file extensions

...

GET /api/categories

POST /api/categories

GET /api/categories/{id}

PUT /api/categories/{id}

DELETE /api/categories/{id}

GET /api/products-in

GET /api/products-out

...

Methods

- PascalCase for public methods
- `Async` suffix on all `async` methods
- Verb-Noun pair: `GetByIdAsync`, `CreateAsync`, `DeleteAsync`

2. Project Structure

...

src/InventoryManagementSystem.Api/

■■■■ Controllers/ # HTTP layer — thin, only routing & status codes

■■■■ Models/ # EF Core entity classes (POCOs)

■■■■ Data/ # ApplicationDbContext + EF Core Migrations

■■■■ DTOs/

■ ■■■■ Requests/ # Input models (Create/Update/Login/Register)

■ ■■■■ Responses/ # Output models (shapes returned to client)

■■■■ Repositories/ # Data access layer (LINQ queries, pagination)

■■■■ Services/ # Business logic layer (orchestrates repos)

■■■■ Helpers/ # Utility/static helper classes

■■■■ wwwroot/

■ ■■■ upload/products/ # Product images
■■■ Program.cs # App entry point & DI registration
■■■ appsettings.json # Configuration
...

Layer responsibilities

| Layer | What it does | What it doesn't do |

|---|---|---|

| **Controller** | Map HTTP verbs to service calls, return status codes | No business logic, no direct DbContext access |

| **Service** | Business rules, stock mutation math, orchestration | No HTTP concerns, no raw SQL |

| **Repository** | EF Core queries, pagination, `Where`/`OrderBy`/`Skip`/`Take` | No business logic, no HTTP |

| **Model** | Entity properties + navigation properties | No behavior, no logic |

| **DTO** | Shape data for input/output | No EF navigation, no circular refs |

Dependency flow

...

Controller → Service (interface) → Repository (interface) → DbContext → SQL Server

↓ ↓

Request DTOs Response DTOs

...

- Controllers depend only on **Service interfaces**
- Services depend only on **Repository interfaces**
- Repositories depend on **DbContext**
- No circular dependencies across layers

3. Code Style

General

- Use `var` when the type is obvious on the right side
- Use expression-bodied members for simple one-liners
- Keep methods under ~20 lines — split into private helpers
- No regions (`#region` / `#endregion`)
- Namespace declarations: **file-scoped** (`namespace X.Y;` not `namespace X.Y { }`)

- Use ``Primary Constructor`` syntax for classes with simple DI (C# 12+)

Async / Await

- ****Go async all the way**** — no `.Result`` or `.Wait()`` calls
- End method names with ``Async`` suffix
- Prefer ``Task`` / ``Task<T>`` over ``void async`` (except event handlers)
- Pass ``CancellationToken`` to EF Core methods (``ToListAsync(ct)``, ``SaveChangesAsync(ct)``)
- ``ConfigureAwait(false)`` in library code

```

```csharp
// Good
public async Task<ProductResponse?> GetByIdAsync(int id, CancellationToken ct = default)
{
 return await _repository.GetByIdAsync(id, ct);
}

// Bad
public ProductResponse? GetById(int id)
{
 return _repository.GetByIdAsync(id).Result;
}
...

```

## ***Error Handling***

- **\*\*No try/catch in controllers\*\*** — use global exception middleware
- Services throw domain exceptions (``KeyNotFoundException``, ``InvalidOperationException``)
- Repository methods let EF Core exceptions bubble up
- ``NotSupportedException`` is placeholder only — replace before merging

## ***Nullability***

- Enable nullable reference types (already set in ``.csproj``)
- Use ``string.Empty`` for non-nullable strings, ``string?`` for optional
- Mark navigation properties as ``null!`` when required by EF but set by runtime

```

```csharp
public Category Category { get; set; } = null!;
...

---

```

4. Git Workflow

Branching

...

main — production-ready, protected

■■■ feature/* — new features (e.g. feature/product-image-upload)

■■■ fix/* — bug fixes (e.g. fix/negative-stock-guard)

■■■ chore/* — tooling, deps, config (e.g. chore/add-guidelines)

...

- Branch off `main`
- Merge back via **squash merge** (keeps history clean)
- Delete branch after merge

Commit Messages

Follow [Conventional Commits](<https://www.conventionalcommits.org/>):

...

<type>(<scope>): <imperative description>

[optional body]

...

| Type | When to use |

|---|---|

| `feat` | New feature |

| `fix` | Bug fix |

| `refactor` | Code change with no behavior change |

| `chore` | Build, deps, tooling |

| `docs` | Documentation only |

| `style` | Formatting, whitespace |

| `test` | Adding/updating tests |

Examples:

...

feat(products): add image upload on create

fix(products-out): prevent negative stock on outbound

refactor(controllers): extract validation to service layer

docs: add project guidelines

...

5. Database / EF Core Conventions

Configuration

- All entity configuration in ``AppDbContext.OnModelCreating()`` using ****Fluent API****
- No data annotations on entity classes (keep POCOs clean)
- Use ``HasColumnType`` for explicit SQL types when default doesn't match

```
```csharp
modelBuilder.Entity<Product>(e =>
{
 e.Property(p => p.Name).HasMaxLength(255);
 e.HasOne(p => p.Category)
 .WithMany(c => c.Products)
 .HasForeignKey(p => p.CategoryId)
 .OnDelete(DeleteBehavior.Cascade);
});
```
```

Timestamps

- ``CreatedAt`` / ``UpdatedAt`` on every entity
- Set in ****Service layer**** (not in `DbContext.SaveChanges` override)
- Always use ``DateTime.UtcNow`` (never local time)

Queries

- Always ``async`` — ``ToListAsync()``, ``FirstOrDefaultAsync()``, ``CountAsync()``
- Use ``Include()`` for eager loading (no lazy loading enabled)
- Write queries in ****Repositories**** only (never in `Controllers` or `Services`)
- Use ``AsNoTracking()`` for read-only queries

Migrations

- Create: ``dotnet ef migrations add <Name>``
- Apply: ``dotnet ef database update``
- Name migrations descriptively: ``AddProductTable``, ``AddCategoryForeignKey``

- Never edit generated migration files — create a new migration instead

6. API Design Conventions

Route Pattern

```
/api/{resource}
/api/{resource}/{id}
/api/{resource}/{id}/{related-resource}
---
```

Pagination

All list endpoints accept:

| Query param | Default | Notes |
|-------------|---------|--------------------------------|
| `page` | `1` | Page number (1-indexed) |
| `perPage` | `10` | Items per page (max 100) |
| `search` | `null` | Full-text search on name/label |

Response shape:

```
```json
{
 "page": 1,
 "perPage": 10,
 "total": 47,
 "lastPage": 5,
 "data": [...]
}
```
```

Standard Response Codes

| Code | When |
|-------|--------------------|
| `200` | Success (GET, PUT) |
| `201` | Created (POST) |

| `204` | No Content (DELETE success) |
| `400` | Bad Request (validation error) |
| `404` | Not Found |
| `401` | Unauthorized (missing/invalid token) |
| `403` | Forbidden (wrong role) |

JSON Naming

- Use **snake_case** for JSON property names via `JsonPropertyName`` attribute on DTOs

```
```csharp
using System.Text.Json.Serialization;

public class CreateProductRequest
{
 [JsonPropertyName("category_id")]
 public int CategoryId { get; set; }
}
```
```

Auth Header

```
```
Authorization: Bearer <token>
```
```

All endpoints except `/api/users/login`` require authentication.

Reminders

- **Run `dotnet build`` before pushing** — no build errors on main
- **Run `dotnet ef migrations list`` before creating a new migration** — ensure you're in sync
- **Don't commit secrets** — connection strings in appsettings are for dev only; use User Secrets or environment variables in production
- **Don't edit generated migration files** — create a new migration to fix mistakes